

CHƯƠNG 05: BÀI TOÁN THỎA MÃN RÀNG BUỘC

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 5

- ◇ 5.1 Định nghĩa CSP (Constraint Satisfaction Problems)
- ◇ 5.2 Suy diễn CSP: Lan truyền Ràng buộc (Constraint Propagation)
- ◇ 5.3 Tìm kiếm Quay lui (Backtracking Search) cho CSP
- ◇ 5.4 Tìm kiếm Cục bộ (Local Search) cho CSP
- ◇ 5.5 Cấu trúc bài toán (The Structure of Problems)

5.1 Định nghĩa CSP

◇ Thay vì coi trạng thái là một "hộp đen" (black box) như các thuật toán tìm kiếm truyền thống, **Bài toán thỏa mãn ràng buộc (CSP)** sử dụng một biểu diễn cấu trúc rõ ràng cho trạng thái.

◇ **Cấu trúc của một CSP bao gồm 3 thành phần:**

- X : Tập hợp các biến (Variables) $\{X_1, X_2, \dots, X_n\}$.
- D : Tập hợp các miền giá trị (Domains) $\{D_1, D_2, \dots, D_n\}$, trong đó D_i là tập các giá trị hợp lệ cho biến X_i .
- C : Tập hợp các ràng buộc (Constraints) giới hạn sự kết hợp các giá trị mà các biến có thể nhận cùng lúc.

◇ Một **Nghiệm (Solution)** của CSP là một phép gán giá trị (Assignment) cho toàn bộ các biến sao cho không vi phạm bất kỳ ràng buộc nào.

Ví dụ: Bài toán Tô màu Bản đồ

- ◇ **Biến:** Các khu vực trên bản đồ Úc
 $X = \{WA, NT, Q, NSW, V, SA, T\}$.
- ◇ **Miền giá trị:** $D_i = \{\text{Red, Green, Blue}\}$.
- ◇ **Ràng buộc:** Không có 2 khu vực kề nhau nào được có cùng màu.

$$C = \{WA \neq NT, WA \neq SA, NT \neq SA, \dots\}$$

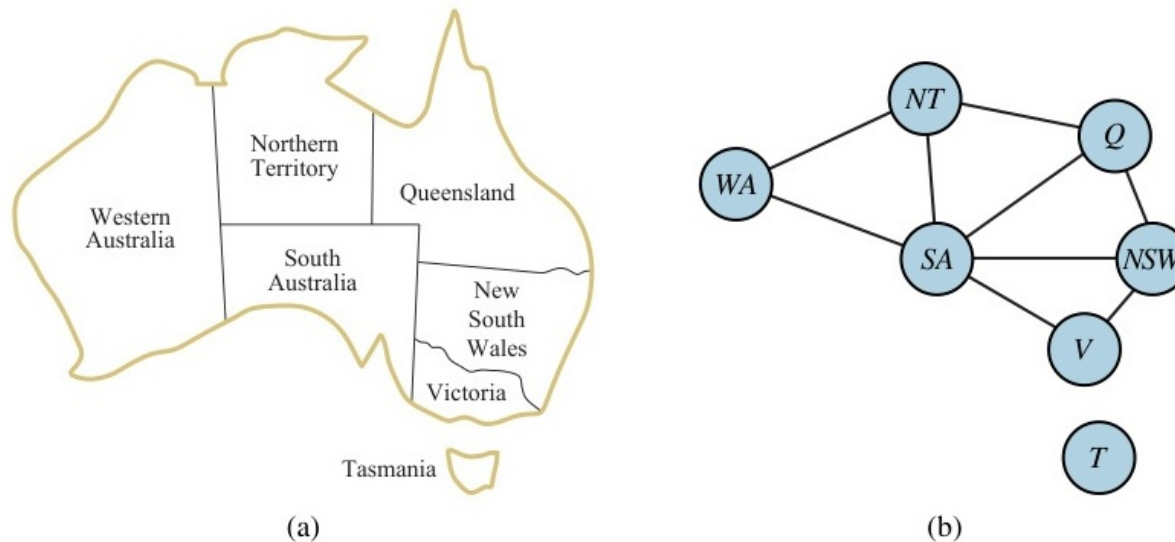


Figure 1: Bản đồ nước Úc và Bài toán Tô màu vùng lãnh thổ.

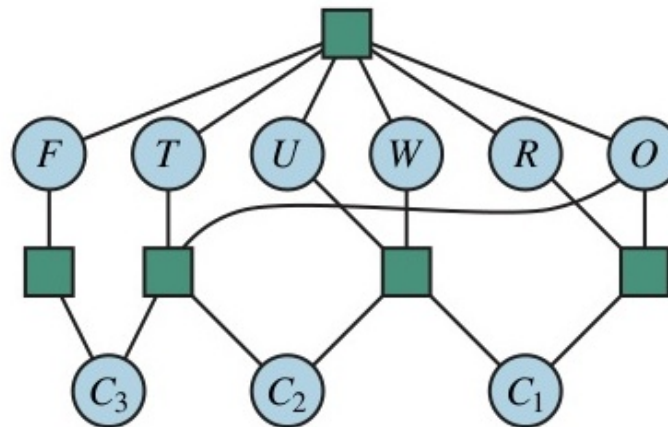
Đồ thị Ràng buộc (Constraint Graph)

◇ CSP thường được biểu diễn trực quan dưới dạng **Đồ thị ràng buộc**:

- **Nút (Node)**: Đại diện cho các biến X_i .
- **Cạnh (Edge)**: Đại diện cho một ràng buộc giữa 2 biến nối với nhau.

$$\begin{array}{cccc}
 & T & W & O \\
 + & T & W & O \\
 \hline
 F & O & U & R
 \end{array}$$

(a)



(b)

Figure 2: Đồ thị ràng buộc tương ứng cho Bài toán Tô màu Bản đồ Úc.

Các loại Ràng buộc

- ◇ **Ràng buộc đơn nguyên (Unary constraint):** Chỉ giới hạn giá trị của 1 biến. (VD: $SA \neq \text{Green}$).
- ◇ **Ràng buộc nhị phân (Binary constraint):** Liên quan đến 2 biến. (VD: $SA \neq WA$). CSP chỉ chứa ràng buộc nhị phân gọi là Binary CSP.
- ◇ **Ràng buộc đa nguyên (Global / N-ary constraint):** Liên quan tới số lượng biến tùy ý. (VD: *Alldiff* - tất cả các biến phải mang giá trị khác nhau như trong bài toán Sudoku).
- ◇ **Gán mềm (Soft constraints):** Không bắt buộc, nhưng tạo ra sự ưu tiên (Preferences). Thường được giải quyết bằng các bài toán tối ưu hóa ràng buộc (Constrained Optimization Problems).

Ví dụ: Ràng buộc đa nguyên (Sudoku)

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Figure 3: Bài toán Sudoku minh họa cho ràng buộc *Alldiff*.

5.2 Suy diễn CSP: Lan truyền Ràng buộc

◇ Không giống tìm kiếm thông thường chỉ biết gán và thử, CSP cho phép **Suy diễn (Inference)** trước khi gán để thu hẹp miền giá trị.

◇ Quá trình này gọi là **Lan truyền ràng buộc (Constraint Propagation)**.

◇ **Tính nhất quán nút (Node Consistency):**

- Một biến đơn lẻ X_i là nhất quán nút nếu mọi giá trị trong miền D_i đều thỏa mãn các ràng buộc đơn nguyên của chính nó.

◇ **Tính nhất quán cung (Arc Consistency):**

- Một cạnh $X_i \rightarrow X_j$ là nhất quán cung nếu với **mọi** giá trị ở D_i , luôn có ít nhất **một** giá trị tương ứng ở D_j thỏa mãn ràng buộc.

Thuật toán AC-3

◇ AC-3 (Arc Consistency 3) là thuật toán kinh điển để đảm bảo tính nhất quán cung trên toàn bộ đồ thị.

◇ **Hoạt động:**

- Đưa tất cả các cung (cạnh có hướng) vào một Hàng đợi (Queue).
- Rút từng cung $X_i \rightarrow X_j$ ra kiểm tra. Loại bỏ các giá trị ở D_i không thỏa mãn ràng buộc với D_j .
- Nếu D_i bị thay đổi, phải đẩy tất cả các cung $X_k \rightarrow X_i$ ngược lại vào hàng đợi để kiểm tra lại hệ quả, vì sự thu hẹp của D_i có thể ảnh hưởng liên đới đến các nút X_k .

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise
queue $\leftarrow$ a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do**
 (*X_i*, *X_j*) $\leftarrow$ POP(*queue*)
 if REVISE(*csp*, *X_i*, *X_j*) **then**
 if size of *D_i* = 0 **then return** false
 for each *X_k* **in** *X_i*.NEIGHBORS - {*X_j*} **do**
 add (*X_k*, *X_i*) to *queue*
return true

function REVISE(*csp*, *X_i*, *X_j*) **returns** true iff we revise the domain of *X_i*
revised $\leftarrow$ false
for each *x* **in** *D_i* **do**
 if no value *y* in *D_j* allows (*x*,*y*) to satisfy the constraint between *X_i* and *X_j* **then**
 delete *x* from *D_i*
 revised $\leftarrow$ true
return *revised*

Figure 4: Thuật toán nhất quán cung AC-3.

5.3 Tìm kiếm Quay lui (Backtracking Search)

- ◇ Nếu Lan truyền rằng buộc không tự thân giải quyết xong bài toán, ta bắt buộc phải **Tìm kiếm (Search)**.
- ◇ Thay vì tạo ra trạng thái gán ngẫu nhiên cho tất cả biến rồi kiểm tra, ta gán từng biến một. Khi phát hiện một biến không còn giá trị hợp lệ, thuật toán sẽ **quay lui (Backtrack)** để sửa lại giá trị của biến gán trước đó.
- ◇ **Lợi ích:** Backtracking với CSP tối ưu và hiệu quả hơn rất nhiều so với Thuật toán tìm kiếm sâu (DFS) vì nó chỉ thám hiểm các phép gán nhất quán.

Thuật toán Tìm kiếm Quay lui

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure  
  return BACKTRACK(csp, { })  
  
function BACKTRACK(csp, assignment) returns a solution or failure  
  if assignment is complete then return assignment  
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp, assignment)  
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do  
    if value is consistent with assignment then  
      add {var = value} to assignment  
      inferences  $\leftarrow$  INFERENCE(csp, var, assignment)  
      if inferences  $\neq$  failure then  
        add inferences to csp  
        result  $\leftarrow$  BACKTRACK(csp, assignment)  
        if result  $\neq$  failure then return result  
        remove inferences from csp  
      remove {var = value} from assignment  
  return failure
```

Figure 5: Một thuật toán quay lui đơn giản cho các bài toán CSP.

Cây Tìm kiếm Quay lui

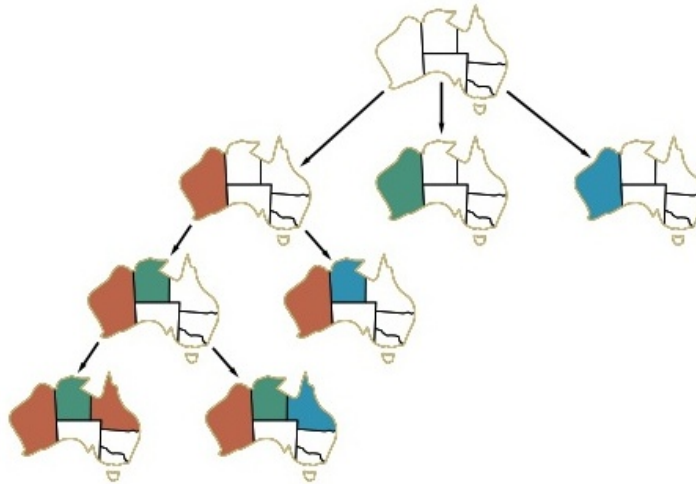


Figure 6: Mô phỏng Tìm kiếm Quay lui qua cây quyết định gán giá trị màu sắc.

Cải thiện Backtracking: Heuristics

- ◇ Backtracking thông thường vẫn khá "mù mờ" nếu chỉ duyệt theo thứ tự ngẫu nhiên. Ta có thể tăng tốc cực lớn bằng cách ưu tiên chọn *biến* và *giá trị* thông minh (Heuristics).
- ◇ **MRV (Minimum Remaining Values)**: Chọn biến nào còn ít giá trị hợp lệ nhất trong miền D để ưu tiên gán trước (Tránh rủi ro phải quay lui hàng loạt về sau).
- ◇ **Degree Heuristic**: Dùng để phá vỡ hòa (tie-breaker) khi nhiều biến bằng điểm MRV. Chọn biến nào có bậc (số lượng liên kết ràng buộc với các biến chưa gán) lớn nhất.
- ◇ **LCV (Least Constraining Value)**: Khi đã chọn được biến, ta nên thử trước giá trị nào ít gây rắc rối (ít bắt buộc thu hẹp giá trị) cho các biến hàng xóm nhất.

Duy trì tính nhất quán trong Tìm kiếm

◇ **Kiểm tra phía trước (Forward Checking):** Tích hợp sẵn suy diễn vào tìm kiếm. Khi gán giá trị cho X , ngay lập tức loại bỏ các giá trị xung đột khỏi miền của các biến kề với X . Nếu có miền của biến nào trở thành Rỗng, lập tức quay lui vì chắc chắn nhánh này vô nghiệm.

◇ **MAC (Maintaining Arc Consistency):** Phiên bản mạnh hơn Forward Checking, gọi lại toàn bộ hàm AC-3 sau mỗi lần gán. Giúp phát hiện sớm lỗi ở xa, tuy tốn thời gian tính toán cho mỗi bước nhưng giúp cắt bỏ được những nhánh duyệt vô ích khổng lồ.

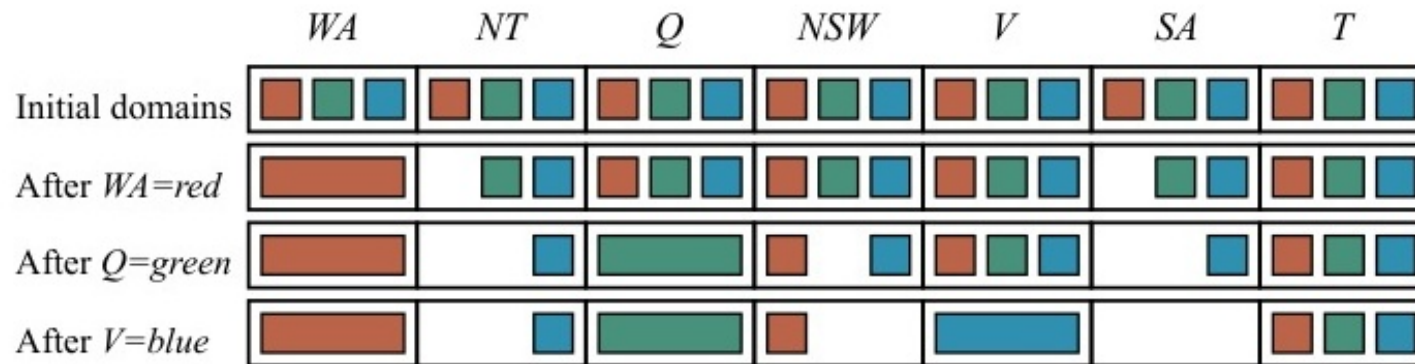


Figure 7: Minh họa quá trình Forward Checking loại bỏ trước các giá trị tương lai.

5.4 Tìm kiếm Cục bộ cho CSP

◇ Các bài toán CSP rất thích hợp để giải quyết bằng các **Thuật toán Tìm kiếm Cục bộ (Local Search)** như leo đồi.

◇ **Quy trình:** Gán ngẫu nhiên giá trị cho toàn bộ các biến (chấp nhận việc vi phạm ràng buộc). Cố gắng thay đổi 1 giá trị mỗi vòng lặp để giảm thiểu số lượng lỗi vi phạm.

◇ **Heuristic Min-Conflicts:**

- Chọn ngẫu nhiên 1 biến đang bị xung đột (vi phạm ràng buộc với biến khác).
- Đổi giá trị của biến đó sang một giá trị mới sao cho **tổng số lượng xung đột** giảm xuống mức thấp nhất.

◇ **Đánh giá khả năng:** Min-Conflicts tỏ ra cực kì mạnh mẽ ở các vấn đề quy mô lớn. Nó có khả năng giải bài toán 1 triệu quân Hậu (1,000,000-Queens) gần như trong thời gian thực!

Thuật toán Min-Conflicts

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or *failure*
inputs: *csp*, a constraint satisfaction problem
 max_steps, the number of steps allowed before giving up

current $\leftarrow$ an initial complete assignment for *csp*
for *i* = 1 to *max_steps* **do**
 if *current* is a solution for *csp* **then return** *current*
 var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES
 value $\leftarrow$ the value *v* for *var* that minimizes CONFLICTS(*csp*, *var*, *v*, *current*)
 set *var* = *value* in *current*
return *failure*

Figure 8: Thuật toán tìm kiếm cục bộ MIN-CONFLICTS cho CSP.

Mô phỏng Min-Conflicts với 8-Queens

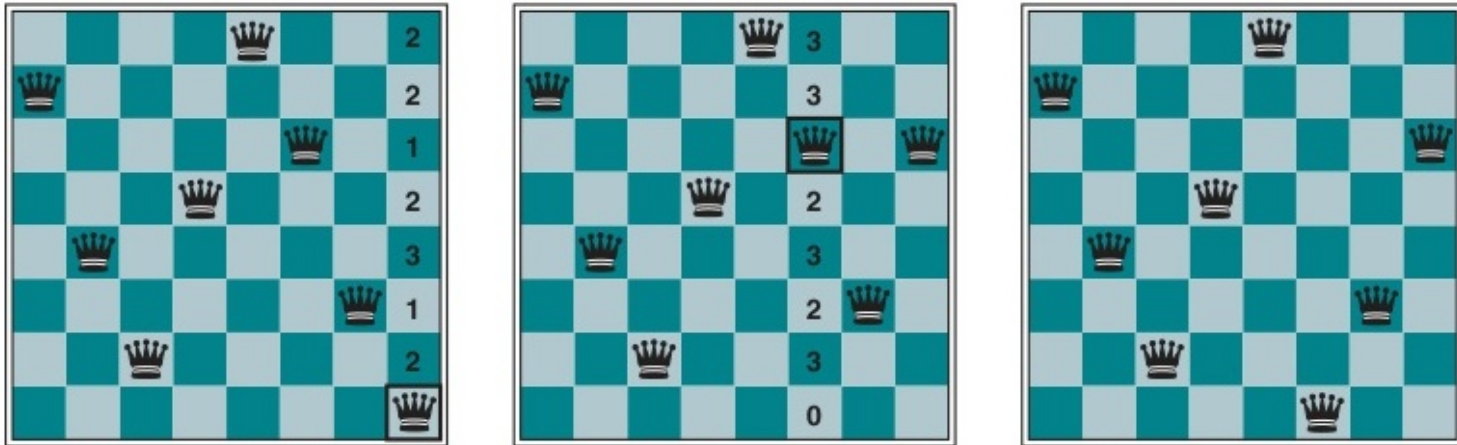


Figure 9: Giải pháp hai bước sử dụng mâu thuẫn tối thiểu cho bài toán 8 con hậu.

5.5 Sức mạnh của Cấu trúc Bài toán

◇ Nếu đồ thị ràng buộc có thể bị chia cắt thành nhiều **Thành phần liên thông rời rạc (Connected components)**, CSP khổng lồ sẽ được chẻ nhỏ thành các bài toán con độc lập. ◇ Khám phá này giúp giảm độ phức tạp từ hàm mũ $\mathcal{O}(d^n)$ xuống còn $\mathcal{O}(c \cdot d^{n/c})$ (Tiết kiệm thời gian kinh hoàng!).

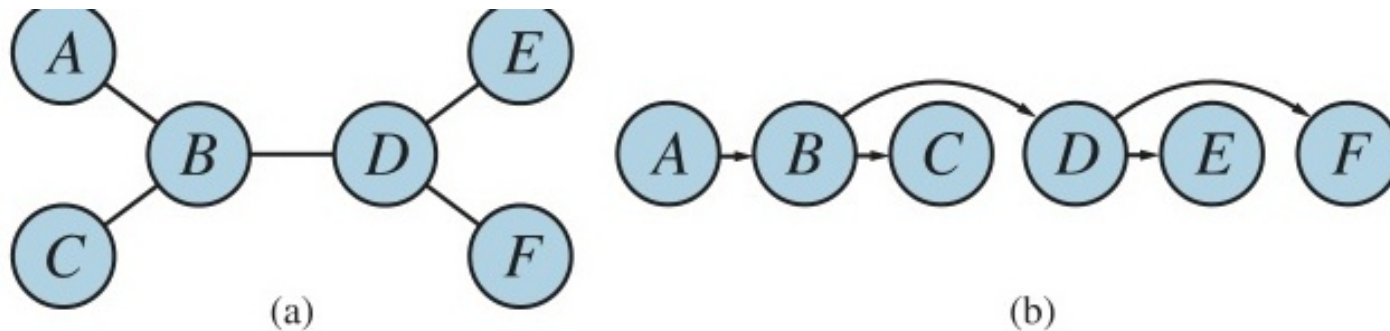


Figure 10: Đồ thị ràng buộc của một CSP có cấu trúc cây.

Giải CSP trên cấu trúc Cây (Tree-structured CSPs)

◇ Nếu đồ thị ràng buộc có dạng **Cây** (không có chu trình khép kín), CSP đó có thể được giải siêu tốc trong thời gian tuyến tính $\mathcal{O}(n \cdot d^2)$!

◇ **Quy trình giải trên cấu trúc Cây:**

- Sắp xếp tuyến tính (Topological sort) các biến sao cho cha đứng trước con.
- Duyệt ngược từ lá lên gốc: Biến đổi để cung Cha $\rightarrow$ Con trở nên nhất quán.
- Duyệt xuôi từ gốc xuống lá: Gán giá trị hợp lệ một cách tự tin mà không bao giờ sợ bị xung đột. (Hoàn toàn không cần quay lui - Backtrack-free!).

Thuật toán Giải CSP trên Cây

function TREE-CSP-SOLVER(*csp*) **returns** a solution, or *failure*
inputs: *csp*, a CSP with components X , D , C

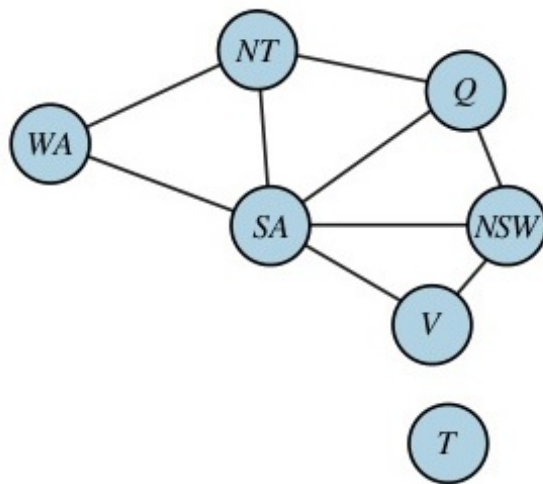
$n \leftarrow$ number of variables in X
assignment $\leftarrow$ an empty assignment
root $\leftarrow$ any variable in X
 $X \leftarrow \text{TOPOLOGICALSORT}(X, \text{root})$
for $j = n$ **down to** 2 **do**
 MAKE-ARC-CONSISTENT(PARENT(X_j), X_j)
 if it cannot be made consistent **then return** *failure*
for $i = 1$ **to** n **do**
 assignment[X_i] $\leftarrow$ any consistent value from D_i
 if there is no consistent value **then return** *failure*
return *assignment*

Figure 11: Thuật toán TREE-CSP-SOLVER để giải quyết các CSP có cấu trúc cây.

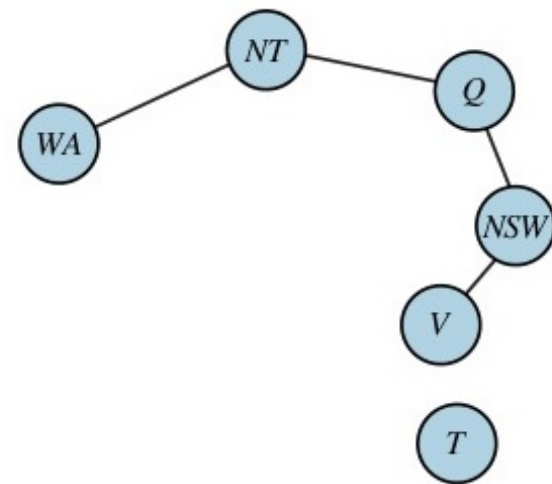
Kỹ thuật Cutset Conditioning

◇ Nếu đồ thị ngoài thực tế có chu trình, ta có thể dùng tiểu xảo **Cutset Conditioning** để loại bỏ chu trình đó:

- Lựa chọn một tập biến nhỏ (gọi là Cutset).
- Gán cứng (instantiate) mọi giá trị có thể cho Cutset đó. Tạm thời xóa các nút này khỏi đồ thị.
- Đồ thị còn lại sẽ bị đứt chu trình và biến về dạng Cây.
- Áp dụng thuật toán giải trên Cây siêu tốc vào từng trường hợp gán của Cutset!



(a)



(b)

Figure 12: Loại bỏ nút trung tâm (Cutset) để bẻ gãy chu trình vòng của đồ thị.

Phân rã Cây (Tree Decomposition)

◇ Mọi đồ thị có thể được phân rã thành một cây gồm các cụm đỉnh (clusters), trong đó mỗi ràng buộc nằm trọn vẹn trong ít nhất một cụm, và mỗi đỉnh tạo thành một cây liên thông giữa các cụm chứa nó.

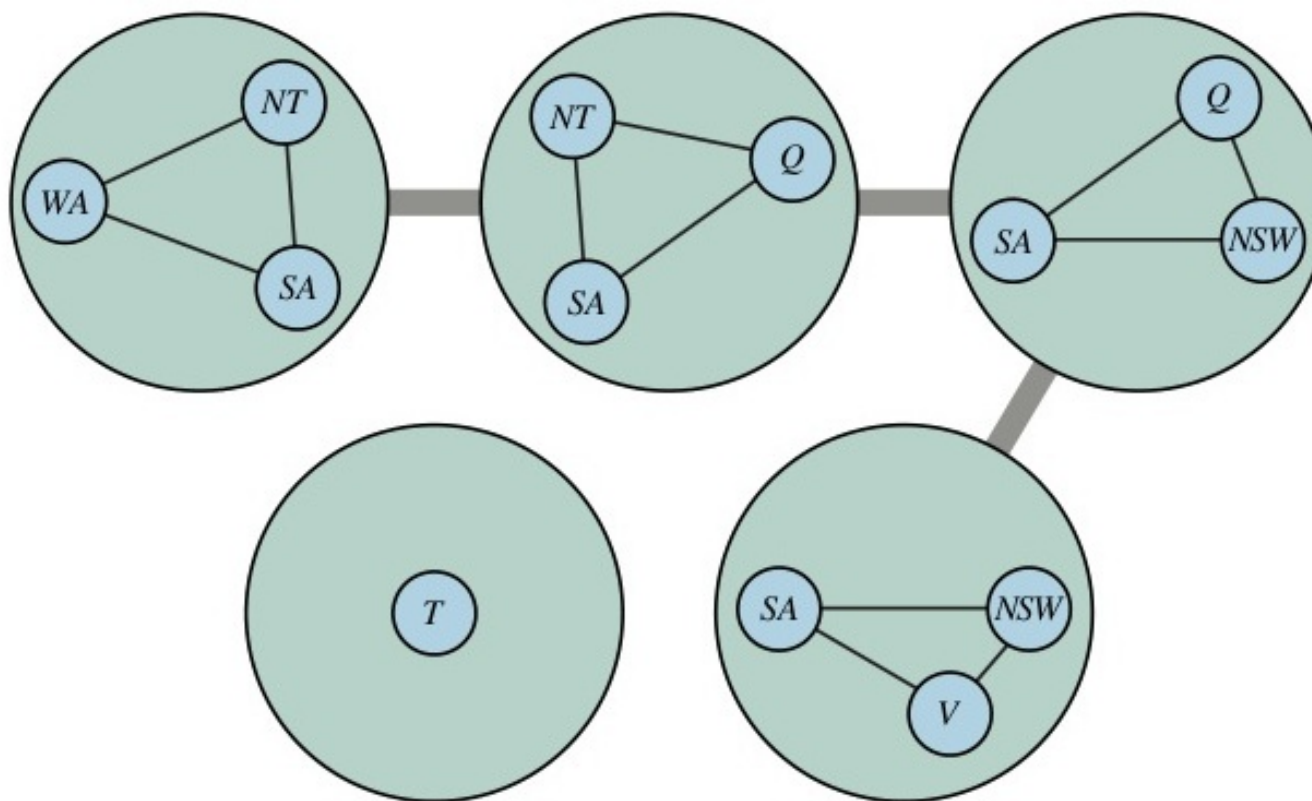


Figure 13: Một phân rã cây (Tree decomposition) của đồ thị ràng buộc bản đồ Úc.

Tóm tắt Chương 5

- ◇ **Định nghĩa cấu trúc:** CSP sử dụng kiến trúc đặc thù bao gồm Biến X , Miền giá trị D và Các quy tắc ràng buộc C .
- ◇ **Tính nhất quán (Consistency):** Sử dụng cơ chế Lan truyền ràng buộc (Đặc biệt là hàm AC-3) giúp thu hẹp miền giá trị sớm để không phải "gán mù".
- ◇ **Quay lui (Backtracking Search):** Cực kì thông minh khi kết hợp chung Heuristics (MRV, Degree, LCV) và kỹ thuật dự báo (Forward Checking, MAC).
- ◇ **Tìm kiếm Cục bộ (Local Search):** Thuật toán Min-Conflicts là ngôi sao sáng cho bài toán phức tạp quy mô lớn.
- ◇ **Tối ưu Cấu trúc đồ thị:** Nếu đưa được bài toán về dạng Cây hoặc Thành phần độc lập rời rạc, thời gian giải CSP sẽ từ mất hàng nghìn năm (Exponential $\mathcal{O}(d^n)$) rút ngắn xuống còn vài giây (Linear $\mathcal{O}(nd^2)$)!